

Architecture: Overpayments & Unallocated Cash

Handling an overpayment is one of the most common edge cases a finance team faces. When a customer pays more than they owe, the system must capture the physical cash while accurately tracking the excess as a liability (or, more specifically, a negative Accounts Receivable balance) owed to the customer.

Let's walk through the exact lifecycle of an overpayment.

1. The Scenario

- **The Debt:** Ivory Blossom owes you **\$100.00** for Sales Invoice `SI202607555` .
- **The Mistake:** Ivory's accounts payable department accidentally writes and sends a check for **\$150.00**.

2. Step 1: Receiving the Cash (The Payment Receipt)

When the finance user receives the \$150 check, they must record exactly what happened in reality. They cannot just enter \$100 to match the invoice; they must enter the full \$150 on the "Create Payment Receipt" screen.

The Automated Journal Entry: When the user hits "Save," the system posts the full amount to the General Ledger:

Description	Account	Debit	Credit
Cash Received	Cash / Bank	\$150.00	
Clear AR	Accounts Receivable (Ivory Blossom)		\$150.00

The System State: Because Ivory originally owed you \$100 (a Debit balance) and you just credited her account for \$150, her total Accounts Receivable balance is now **-\$50.00** (a Credit balance). *You essentially owe her money.*

3. Step 2: The Allocation Phase (The Magic of "Unapplied")

This is where the `allocations` database table (from our `customer_outstanding_logic.md` architecture) goes to work.

On the Payment Receipt UI, the user allocates the funds against the open invoice:

- **Total Receipt Amount:** \$150.00
- **Allocated to** `SI202607555` : \$100.00
- **Unallocated Amount:** \$50.00

The system automatically flags this \$50.00 as "Payment on Account" or "Unallocated Cash." It sits on Ivory's profile as an available credit.

4. Step 3: Resolving the Unallocated Cash

The \$50 cannot sit there forever. The business has two paths to clear this balance, and your system must support both.

Path A: Apply to a Future Invoice (Netting)

Ivory realizes her mistake and says, *"Keep the \$50; I'll use it for my next order."*

1. Next week, Ivory buys goods worth **\$75.00** (creating a new invoice).
2. Your Finance user goes to the **Credit Application** screen (the exact same UI logic we just reviewed for Sales Returns!).
3. Instead of selecting a Credit Note, they select the **Unallocated Payment Receipt**.
4. They apply the \$50 unallocated cash to the new \$75 invoice, lowering the remaining debt to \$25. No cash changes hands.

Path B: The Cash Refund

Ivory demands her \$50 back immediately.

1. Your Finance user opens the **Create Customer Refund** screen.
2. They select Ivory Blossom.
3. The grid fetches the \$50 "Unallocated Cash" balance from that specific Payment Receipt.
4. They process the refund.

The Automated Refund Journal Entry:

Description	Account	Debit	Credit
Clear the Credit	Accounts Receivable (Ivory Blossom)	\$50.00	
Cash Paid Out	Cash / Bank		\$50.00

Result: Her AR balance goes back to \$0.00, and your physical cash decreases by \$50.

5. UI Requirements for Overpayments

To make this seamless for the Finance team, your **Payment Receipt** screen needs three specific UI elements:

1. **Receipt Amount Field:** Where the user types `$150.00`.
2. **Allocation Grid:** Where the user types `$100.00` next to the specific invoice.
3. **A Dynamic Footer:** A live-calculating footer that clearly displays:
 - *Total Applied:* \$100.00
 - *Unallocated / To Account:* **\$50.00** (This gives the user confidence they are logging the overpayment correctly before they hit save).